

LAPORAN TUGAS KECIL

IF2211 Strategi Algoritma

Semester II tahun 2025/2026

Penyelesaian Permainan *Queens* Linkedin

Dipersiapkan oleh:

Benedict Darrel Setiawan – 13524057

**Sekolah Teknik Elektro dan Informatika - Institut Teknologi
Bandung**

Jl. Ganesha 10, Bandung 40132

1. Pendahuluan

Queens adalah gim logika yang tersedia pada situs jejaring profesional LinkedIn. Tujuan dari gim ini adalah menempatkan queen pada sebuah papan persegi berwarna sehingga terdapat hanya satu queen pada tiap baris, kolom, dan daerah warna. Selain itu, satu queen tidak dapat ditempatkan bersebelahan dengan queen lainnya, termasuk secara diagonal.

Fungsi dari program yang dirancang adalah untuk mencari sebuah solusi dari suatu game Queens dengan menggunakan aplikasi algoritma Brute Force. Algoritma Brute Force sendiri merupakan sebuah algoritma yang akan mencari semua kemungkinan hingga menemukan satu solusi yang valid.

2. Penjelasan Algoritma

Pseudocode untuk algoritma Brute Force yang digunakan di dalam program ini adalah sebagai berikut

```
procedure SolveBoard(input/output B : Board)
{ Mencari solusi dari sebuah game Queens sesuai dengan Board B
  Masukan : Board B yang valid dan belum terselesaikan
  Luaran : Board B yang sudah terselesaikan }
```

Algoritma :

```
while not(noPossibilityLeft()) and not(allQueensPlaced()) do
  while not(noTilesLeft()) and not(allQueensPlaced()) do
    if (isQueenPlaceable()) then
      placeQueen()
      setSelectedTile(0,0)
    else
      moveSelectedTile()
  if not(allQueensPlaced()) then
    backtrackToLastQueen()
```

Penjelasan atas setiap fungsi dan prosedur yang digunakan adalah sebagai berikut

- noPossibilityLeft*()
Fungsi yang mengembalikan boolean. True jika sudah tidak ada kemungkinan yang dapat dicari lagi. False jika masih ada. Memiliki kompleksitas waktu $O(1)$
- allQueensPlaced*()
Fungsi yang mengembalikan boolean. True jika semua queen sudah diletakkan di board, atau dalam kata lain, jika banyaknya queen di board sama dengan dimensi board. False jika sebaliknya. Memiliki kompleksitas waktu $O(1)$
- noTilesLeft*()
Fungsi yang mengembalikan boolean. True jika selected tile sudah mencapai akhir board, sehingga tidak dapat meletakkan queen lagi. False jika sebaliknya. Memiliki kompleksitas waktu $O(1)$
- isQueenPlaceable*()
Fungsi yang mengembalikan boolean. True jika queen dapat diletakkan di selected tile, yaitu saat selected tile tidak memiliki column, row, dan warna

yang sama dengan sebuah queen yang sudah diletakkan serta tidak bersentuhan secara diagonal dengan queen lain. False jika tidak. Memiliki kompleksitas waktu $O(n)$ dengan n merupakan dimensi board.

- e. *placeQueen()*
Prosedur yang meletakkan queen di board. Memiliki kompleksitas $O(1)$
- f. *setSelectedTile(input x: integer, input y: integer)*
Prosedur yang mengubah koordinat selected tile di board menjadi (x,y). Memiliki kompleksitas waktu $O(1)$
- g. *moveSelectedTile()*
Prosedur yang menggeser koordinat selected tile sebanyak 1 kali di sumbu x. Jika selected tile sudah mencapai ujung sumbu x, maka tile akan menggeser di sumbu y sebanyak 1 kali dan mengubah koordinat sumbu x ke 0. Memiliki kompleksitas waktu $O(1)$
- h. *backtrackToLastQueen()*
Prosedur yang menghapus queen terakhir yang diletakkan dan melakukan *moveSelectedTile()* sebanyak 1 kali. Memiliki kompleksitas waktu $O(1)$

Untuk menentukan kompleksitas algoritma keseluruhan, algoritma akan perlu dijelaskan lebih lanjut. Algoritma akan pertama-tama mengecek apakah sebuah queen dapat diletakkan di selected tile saat ini. Hal ini dilakukan sebanyak n^2 kali dengan n merupakan dimensi dari board karena board memiliki luas $n \times n$. Sesuai dengan kompleksitas waktu setiap fungsi dan prosedur yang digunakan dalam proses ini maka kompleksitas waktu akan berupa $n^2 \times (O(n) + O(1) + O(1) + O(1) + O(1))$ yang setara dengan $O(n^3)$.

Karena, queen yang harus diletakkan sebanyak n , maka minimal proses ini harus dilakukan sebanyak n kali. Namun, dikarenakan dapat terjadinya backtracking, ada kondisi dimana algoritma akan menjalankan proses sebanyak $(n^2)!/(n^2 - n)!$ atau $(n^2)(n^2 - 1) \dots (n^2 - n)$, disebabkan setiap terletaknya queen terdapat sebanyak n^2 jalur yang mungkin diambil yang semakin berkurang 1 untuk n buah queen. $(n^2)(n^2 - 1) \dots (n^2 - n)$ setara dengan $O(n^{2n})$ Jika digabungkan dengan proses sebelumnya, kompleksitas waktu seluruh algoritma akan berupa $O(n^3 \cdot n^{2n})$.

3. Perancangan Program

Program Queen Solver ini dibangun menggunakan bahasa pemrograman Java. Hal ini dilakukan agar memudahkan atribut-attribut dari game seperti board dan tile. Source code dari program ini dibagi menjadi tiga yaitu, modules, controllers, dan helper.

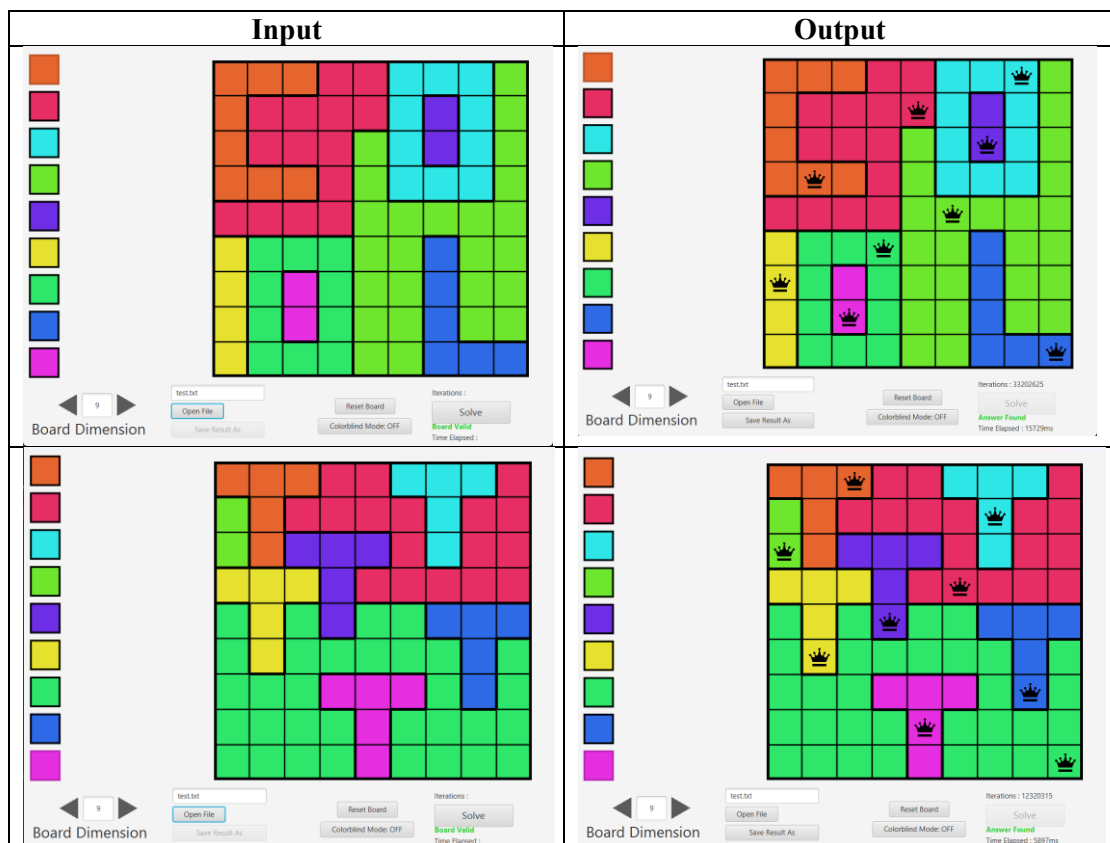
a. modules

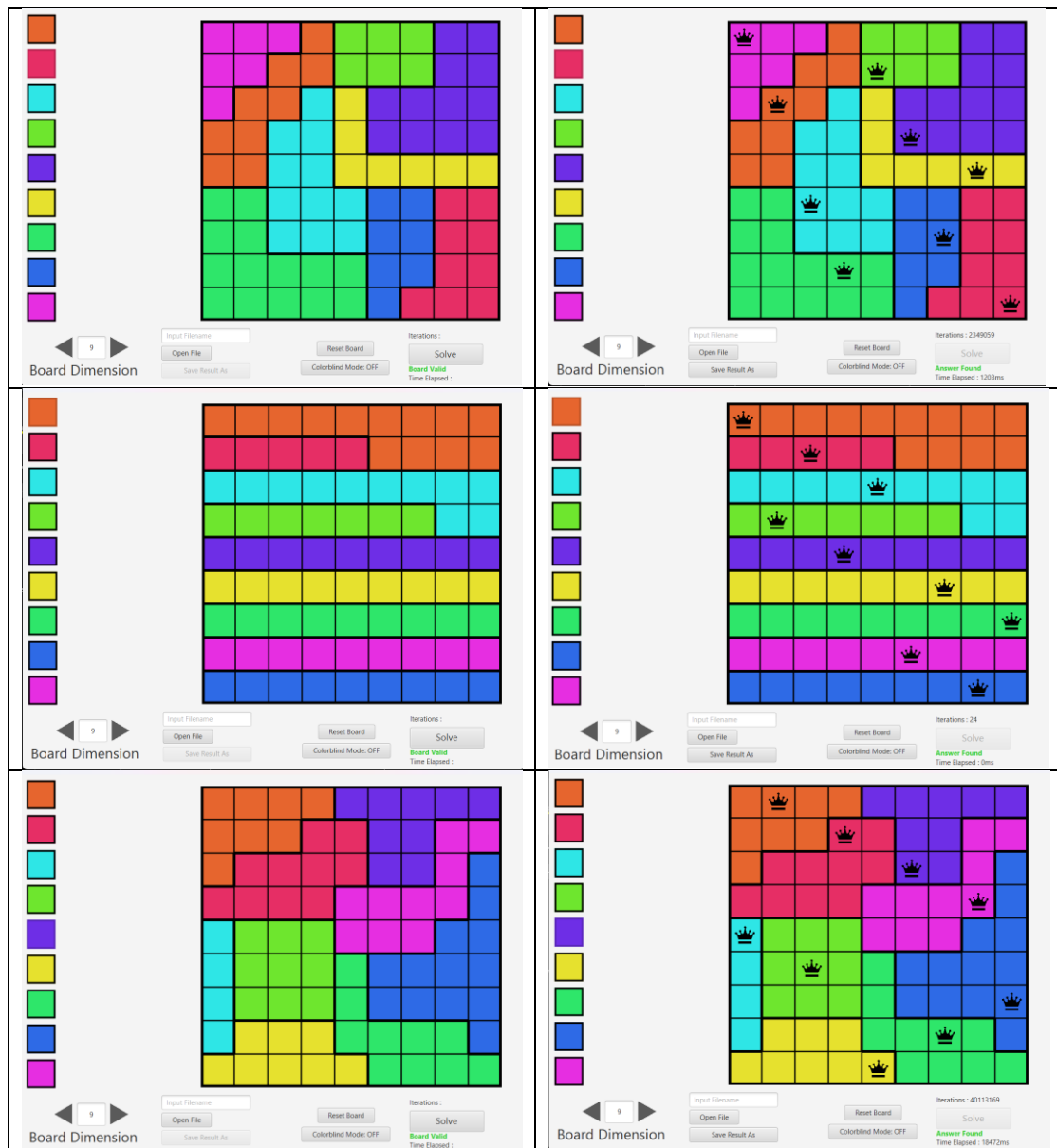
Class-class yang terkandung di dalam modules merupakan logika dasar untuk merancang algoritma pencarian solusi. Berikut adalah penjelasan singkat terkait class yang terkandung di dalam modules.

- **Tile**
Mengatur atribut tile yaitu koordinat x dan y, serta warna atau karakter. Method yang terkandung dalam class ini dikhususkan untuk perbandingan atribut dari dua object tile.
- **Board**
Board merupakan class yang terdiri dari matriks dari object tile, array 1D object tile yang bertujuan untuk melakukan tracking queen. Method yang terkandung dalam class ini berfungsi untuk membuat, mengatur, dan mendapatkan state dari Board.

- **Solver**
Solver merupakan class yang terdiri dari Board. Class ini berfungsi sebagai algoritma-algoritma yang membangun algoritma utama pencarian solusi board.
- b. **controllers**
Class-class yang terkandung di dalam controllers mengatur alur dari seluruh program termasuk antarmuka yang ditampilkan. Berikut adalah penjelasan singkat terkait class yang terkandung di dalam controllers.
 - **BoardController**
BoardController merupakan class utama yang mengatur tampilan dan interaksi antarmuka.
 - **SolverTask**
SolverTask merupakan class yang mengandung algoritma pencarian utama dari program. Di class ini diterapkan multithreading agar mendapatkan antarmuka yang responsive selagi berjalannya algoritma.
- c. **helper**
Class-class yang terkandung di dalam helper membantu membuat dan menyimpan hal-hal yang tidak terkait dengan algoritma utama program.
 - **ColorPicker**
ColorPicker membantu menentukan warna dari tile yang berada di board dan menyimpannya di dalam sebuah map dengan karakter tile sebagai key, dan nilai hue sebagai value.

4. Pengujian Program





5. Lampiran

Repository proyek : https://github.com/BenedictD-RH/Tucil1_13524057

No	Poin	Ya	Tidak
1.	Program berhasil di kompilasi tanpa kesalahan	✓	
2.	Program berhasil di jalankan	✓	
3.	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4.	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5.	Program memiliki Graphical User Interface (GUI)	✓	
6.	Program dapat menyimpan solusi dalam bentuk file gambar	✓	

6. Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

A handwritten signature in black ink, consisting of stylized, overlapping loops and a vertical line extending downwards.

Benedict Darrel Setiawan